

Guía 2 Spring-boot

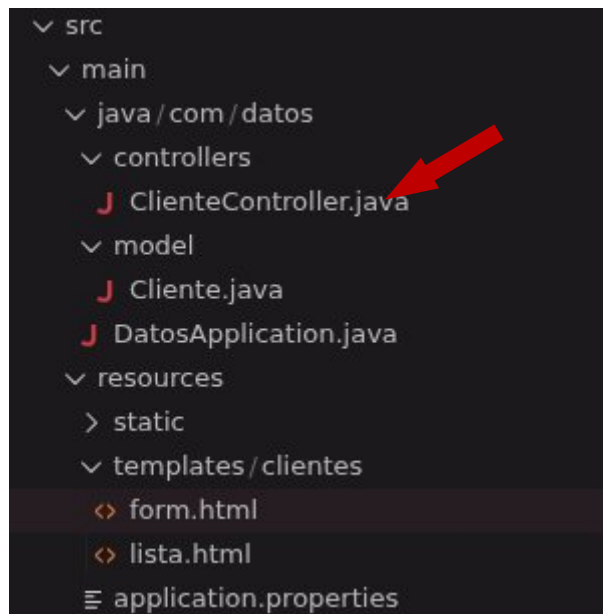
Lenguaje : Java 21

Nombre : Miguel Abraham Lacruz Medina

Ficha : 3114733

Fecha de creación : Jueves 26 Febrero

Creación del controlador **ClienteController.java**



Le agregamos el siguiente contenido

```
package com.datos.controllers;

import java.util.ArrayList;
import java.util.List;

import org.springframework.stereotype.Controller;
import org.springframework.ui.Model;
import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.PostMapping;
import org.springframework.web.bind.annotation.RequestParam;

import com.datos.model.Cliente;

@Controller
public class ClienteController {
    // Crear la lista.
    private static List<Cliente> lista = new ArrayList<>();
    // Crear el contador.
    private static long contador;
    @GetMapping("/clientes")
    public String Index(Model model){
        /* -----
        Metodo 1 para agregar valores a la lista, pero no es dinamico si no que es **estatico**
        es decir se hace mediate el codigo y estos valores son fijos y no responden a un metodo POST.
        ----- */

        List<Cliente> lista = new ArrayList<>();
        lista.add(new Cliente(1L, "Miguel", 123));
        lista.add(new Cliente(2L, "Miguel 1", 456));
        lista.add(new Cliente(3L, "Miguel 2", 789));
        model.addAttribute("Cliente", lista);
    }
}
```

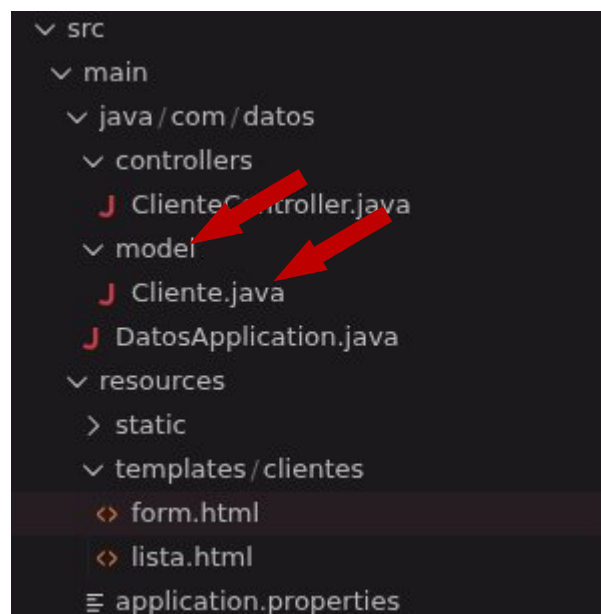
```

-----
Metodo 2 para agregar nuevos clientes a la lista, este bloque de codigo verificar si la lista esta vacia
si la lista esta vacia, te redirige a la vista "clientes/nuevo"
si NO esta vacia guarda los valores y los manda al modelo para adespues poderlos imprimir en forma de lista
para despues llevarte a la vista "clientes/lista".
----- */
if (lista.isEmpty()){
    // Redirige a la vista clientes/nuevo
    return "redirect:/clientes/nuevo";
} else {
    // Agrega los valores de la lista y los manda al modelo.
    model.addAttribute(attributeName: "Cliente", lista);
    // Redirige a la vista clientes/lista.
    return "clientes/lista";
}
}
// Metodo Get.
@GetMapping("/clientes/nuevo")
public String Agregar(){
    return "clientes/form";
}
// Metodo Post.
@PostMapping("/clientes/guardar")
public String GuardarCliente(@RequestParam String nombre,@RequestParam String telefono){
    Cliente nuevo = new Cliente(++contador, nombre, telefono);
    lista.add(nuevo);
    return "redirect:/clientes";
}
}

```

Lo que hace el código es : Primero verificar si la lista tiene contenido, si no tiene contenido el código retorna al usuario a la vista **cliente/nuevo** para agregar un nuevo cliente.

Creamos la carpeta **model** y el modelo llamado **Cliente.java**



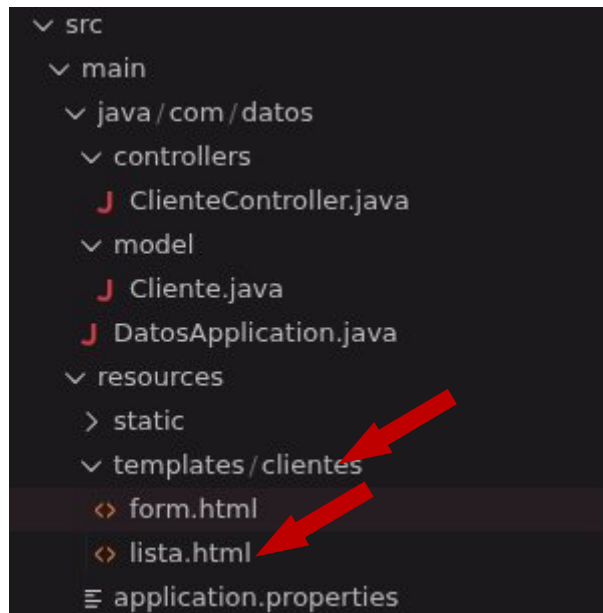
Y se le agrega la siguiente estructura

```
package com.datos.model;

public class Cliente {
    // Id
    private Long id;
    // Nombre
    private String nombre;
    // Telefono
    private String telefono;
    // Crear constructor vacio para thymeleaf
    public Cliente(){}
    // Constructor que inicializa las variables antes creadas.
    public Cliente(Long id, String nombre, String telefono){
        this.id = id;
        this.nombre = nombre;
        this.telefono = telefono;
    }
    // Metodo Get que retorna el id
    public Long getId(){
        return id;
    }
    // Metodo Set que guarda informacion del id
    public void setId(Long id){
        this.id = id;
    }
    // Metodo Get que retorna el nombre
    public String getNombre(){
        return nombre;
    }
    // Metodo Set que guarda informacion del nombre
    public void setNombre(String nombre){
        this.nombre = nombre;
    }
    // Metodo Get que retorna el telefono
    public String getTelefono(){
        return telefono;
    }
    // Metodo Set que guarda la informacion del telefono
    public void setTelefono(String telefono){
        this.telefono = telefono;
    }
}
```

Esta es la estructura básica para el modelo con 3 atributos : 1) **id**, 2) **nombre**, 3) **teléfono**, los cuales son inicializados en el constructor.

Creamos la carpeta **clientes** en **templates**, dentro de la nueva carpeta **clientes** creamos la vista **lista.html**

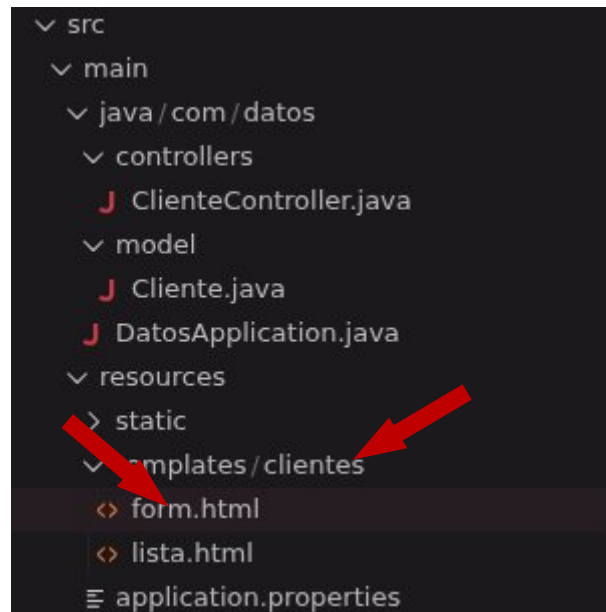


Con el siguiente contenido para la vista

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Lista de clientes</title>
</head>
<body>
  <!-- Boton que te lleva a la vista del formulario -->
  <a th:href="@{/clientes/nuevo}"> Agregar clientes</a>
  <h1>Clientes</h1>
  <table>
    <thead>
      <tr>
        <th>ID</th>
        <th>Nombre</th>
        <th>Telefono</th>
      </tr>
    </thead>
    <tbody>
      <!-- Recorre Cliente que viene del controlador ClienteController -->
      <tr th:each="c : ${Cliente}">
        <!-- Muestra en pantalla el contenido de cada uno -->
        <td th:text="${c.id}"> </td>
        <td th:text="${c.nombre}"> </td>
        <td th:text="${c.telefono}"> </td>
      </tr>
    </tbody>
  </table>
</body>
</html>
```

Lo que hace el código es tener una lista con una cabecera que contiene : **ID**, **Nombre**, **Teléfono**, después en el cuerpo este contiene un **tr** que este es el que itera dentro del modelo que se envía desde el controlador **ClienteController.java**

Creamos la vista **form.html** dentro de la carpeta **clientes** para el formulario

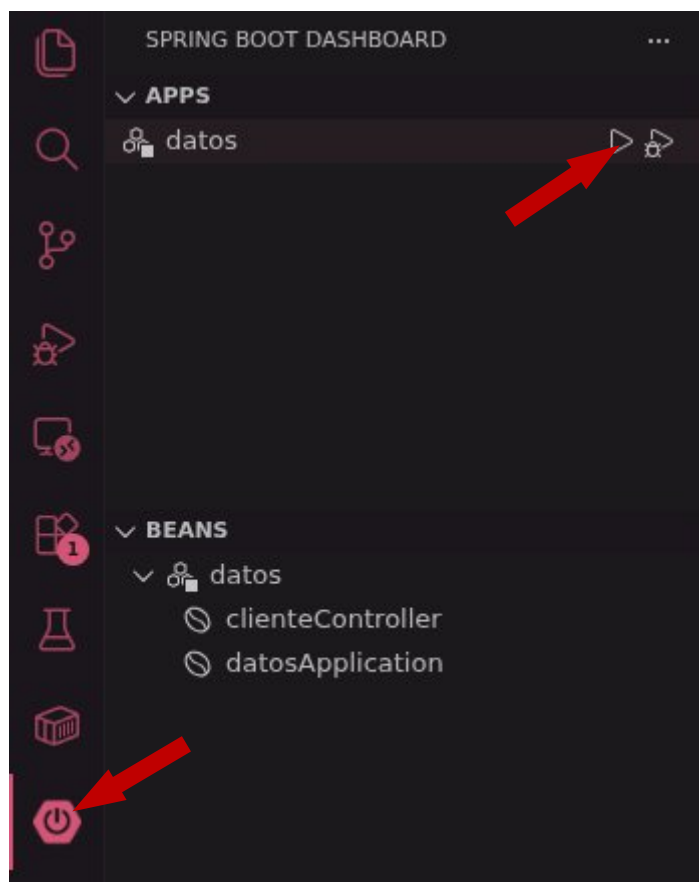


Con la que se le agrega la siguiente estructura

```
<!DOCTYPE html>
<html lang="es">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title> Formulario - agregar clientes </title>
</head>
<body>
  <!-- Boton que te lleva a la vista de lista de clientes --->
  <a th:href="@{/clientes}"> Ver clientes </a>
  <h1>Agregar clientes</h1>
  <!-- formulario que guarda los valores post --->
  <form th:action="@{/clientes/guardar}" method="POST">
    <input type="text" placeholder="Nombre" name="nombre" required> <br>
    <input type="number" placeholder="Telefono" name="telefono" required> <br>
    <button type="submit"> Enviar </button>
  </form>
</body>
</html>
```

Esta vista solo contiene 4 cosas : 1) enlace de regreso a la vista **clientes**, 2) input de tipo texto para el Nombre, 3) input del tipo **number** para el **Teléfono**, 4) **button** de tipo **submit** para enviar los datos al modelo, estos 3 ultimos estando dentro de la etiqueta **form** para el formulario.

Encendemos el servidor dirigiéndonos al icono de encendido y para despues presionar el **triangulo** para correr el servidor.



Verificamos que el servidor este funcionando correctamente, en este caso esta corriendo en el puerto **8080**.

```

:: Spring Boot :: (v4.0.3)

2026-02-26T05:07:58.141-05:00 INFO 76557 --- [datos] [ restartedMain] com.datos.DatosApplication : Starting DatosApplication using Java 21.0.10 with PID 76557 (/home/shaggy/ProyectosJava/datos/target/classes started by shaggy in /home/shaggy/Documentos/ProyectosJava/datos)
2026-02-26T05:07:58.202-05:00 INFO 76557 --- [datos] [ restartedMain] com.datos.DatosApplication : No active profile set, falling back to 1 default profile: "default"
2026-02-26T05:07:58.928-05:00 INFO 76557 --- [datos] [ restartedMain] .e.DevToolsPropertyDefaultsPostProcessor : Devtools property defaults active! Set 'spring.devtools.add-properties' to 'se' to disable
2026-02-26T05:07:58.928-05:00 INFO 76557 --- [datos] [ restartedMain] .e.DevToolsPropertyDefaultsPostProcessor : For additional web related logging consider setting the 'logging.level.w
2026-02-26T05:08:05.691-05:00 INFO 76557 --- [datos] [ restartedMain] o.s.boot.tomcat.TomcatWebServer : Tomcat initialized with port 8080 (http)
2026-02-26T05:08:05.762-05:00 INFO 76557 --- [datos] [ restartedMain] o.apache.catalina.core.StandardService : Starting service [Tomcat]
2026-02-26T05:08:05.772-05:00 INFO 76557 --- [datos] [ restartedMain] o.apache.catalina.core.StandardEngine : Starting Servlet engine: [Apache Tomcat/11.0.18]
2026-02-26T05:08:05.992-05:00 INFO 76557 --- [datos] [ restartedMain] b.w.c.s.WebApplicationContextInitializer : Root WebApplicationContext: initialization completed in 7042 ms
2026-02-26T05:08:08.671-05:00 INFO 76557 --- [datos] [ restartedMain] o.s.boot.tomcat.TomcatWebServer : Tomcat started on port 8080 (http) with context path '/'
2026-02-26T05:08:08.724-05:00 INFO 76557 --- [datos] [ restartedMain] com.datos.DatosApplication : Started DatosApplication in 13.285 seconds (process running for 17.566)

```


Abrimos nuestro navegador y entramos al enlace **localhost:8080/clientes** (el puerto puede ser diferente dependiendo de como hemos configurado el puerto del servidor).

Como lo hemos programado anteriormente, si la lista esta vacía nos lleva automáticamente a la vista **clientes/nuevo**.



[Ver clientes](#)

Agregar clientes

Nombre
Telefono
Enviar

Ingresamos valores para nuestra lista para después presionar **Enviar** para agregar los atributos al modelo.



[Ver clientes](#)

Agregar clientes

Miguel
1234567890
Enviar

Después de presionar **Enviar** seremos enviados a la vista **clientes/lista**, mostrando que nuestra pequeña “**base de datos**” funciona correctamente.



[Agregar clientes](#)

Cientes

ID	Nombre	Telefono
----	--------	----------

1	Miguel	1234567890
---	--------	------------